



Secretlab

Tech Exercise

In order to avoid any misunderstandings for both parties in the future, you should proceed with this exercise only if you agree with all the statements below:

- 1. You agree and recognise that Secretlab may already be independently working on the same project. You agree that there is a likelihood that Secretlab's employees or contractors may author, create or develop the same or similar design or content as covered by your submission now or in the future, and, therefore, you agree that any similarity between Secretlab's products and your submission is merely coincidental and cannot serve as circumstantial evidence of any misappropriation.*
- 2. You will not submit to us any information in breach of any confidentiality, or any contractual obligations you have towards others.*
- 3. You agree that no contractual or working relationship is being created between you and Secretlab solely by your participation in this exercise.*
- 4. You agree that any materials provided for this exercise are to be kept strictly private and confidential.*



Build a version-controlled key-value store with an HTTP API we can query that from.
The API needs to be able to:

1. Accept a key(string) and value(some JSON blob/string) and store them.
2. Accept a key and return the corresponding latest value
3. When given a key AND a timestamp, return whatever the value of the key at the time was.
4. Displays all values currently stored in the database.

Assume only GET and POST requests for simplicity.

All timestamps are UNIX timestamps according to the UTC timezone.

Example:

Method: POST

Endpoint: /object

Body: JSON: {mykey : value1}

Time: 6pm

Method: GET

Endpoint: /object/mykey

Response: value1 object

Method: POST

Endpoint: /object

Body: JSON: {mykey : value2}

Time: 6.05 pm



Method: GET

Endpoint: /object/mykey

Response: value2 object

Method: GET

Endpoint: /object/mykey?timestamp=1440568980 [6.03pm]

Response: value1 object

Method: GET

Endpoint: /object/get_all_records

Response: Returns JSON Array of all records data and their values currently stored in the DB

Guidelines:

1. Use PHP as the language for development (Laravel framework).
2. Data should be stored in a database service of your choice e.g (MySQL, MongoDB, etc).
3. Unit testing and CI/CD service should monitor the master branch for updates. E.g any change to a file on the master branch should trigger a build that will initiate testing and deploy to the hosting server if there are no errors.
4. Deploy this to a server and give us the URL.
5. Push the code to a public git repository and give us the URL.
6. You are free to use any open-source database/ hosting/ CICD/ testing service.



A good submission will have the following characteristics:

1. Clean and extendable code
2. Handle a wide variety of edge cases
3. Not break under a reasonable load
4. A good unit testing coverage report
5. Build status for each commit to show test passing status

***Bonus** - Add code coverage reporting into CI/CD pipeline

Candidates are welcome to use AI tools as an aid, however this must be declared in the read me document. AI tools are only as good as the Engineer guiding it – this means that **you** are (a) ultimately responsible for and (b) judged based on your command of AI in delivering the submission.

We are NOT judging for algorithmic brilliance or cool one-liners. We want to see if you can write production-quality code. Please build this assuming you are building a production-quality service.

While some of the above requirements may be new to you, we would like to see your effort to accomplish the task.